**Universidad Externado de Colombia**

Departamento de Matemáticas

Machine Learning II

Proyecto Integrador — Segunda Entrega

Fase 2: datos, modelamiento y validación

---

# Predicción de deserción estudiantil en el Tecnológico de Monterrey mediante modelado avanzado de Machine Learning

---

**Integrantes**

Miguel Camargo
Camilo Hernández

**Profesor:** Wilmer Pineda

2 de octubre de 2026

Este informe corresponde a la segunda fase del proyecto: datos, modelamiento y validación. La pregunta es qué modelo predice mejor si un estudiante de pregrado dejará la universidad durante su primer año, usando lo que se sabe de él al terminar su primer semestre. Si la predicción es buena, la universidad puede ofrecer acompañamiento a tiempo a quienes más lo necesitan. El código y los pasos para repetir cada resultado se describen en el anexo.

## 1. Consolidación de los datos

**Fuente.** Se usó el *Student Dropout Dataset* del Tecnológico de Monterrey (Alvarado-Uribe et al., 2022), obtenido por solicitud formal al IFE Data Hub. Tiene 143.326 registros de estudiantes de bachillerato y pregrado que ingresaron entre agosto de 2014 (cohorte AD14) y agosto de 2020 (cohorte AD20). Son datos reales latinoamericanos, así que no fue necesario simular.

**Población de estudio y variable objetivo.** Villar y Andrade (2024) predicen la deserción con dos tipos de información: la que se conoce al ingresar y el desempeño en los primeros semestres. En este dataset, las notas del primer semestre (promedio, materias reprobadas y materias retiradas) solo existen para las cohortes AD19 y AD20 (Figura 1). Por eso el estudio se concentra en los 24.251 estudiantes de pregrado de esas dos cohortes. La predicción se hace al terminar el primer semestre, y la variable objetivo vale 1 si el estudiante ya no está matriculado al cerrar su primer año. Se dejó por fuera a los 256 estudiantes que se retiraron a mitad del primer semestre: cuando se hace la predicción ya se fueron, y su promedio es bajo porque abandonaron, no al revés. El artículo distingue tres desenlaces (graduado, rezagado y desertor), pero este dataset no registra la graduación, así que el problema tiene dos clases: deserta o no. Solo deserta el 7,7 % de los estudiantes, frente al 32 % del artículo, y esa diferencia pesa en la elección de métricas (Sección 4).

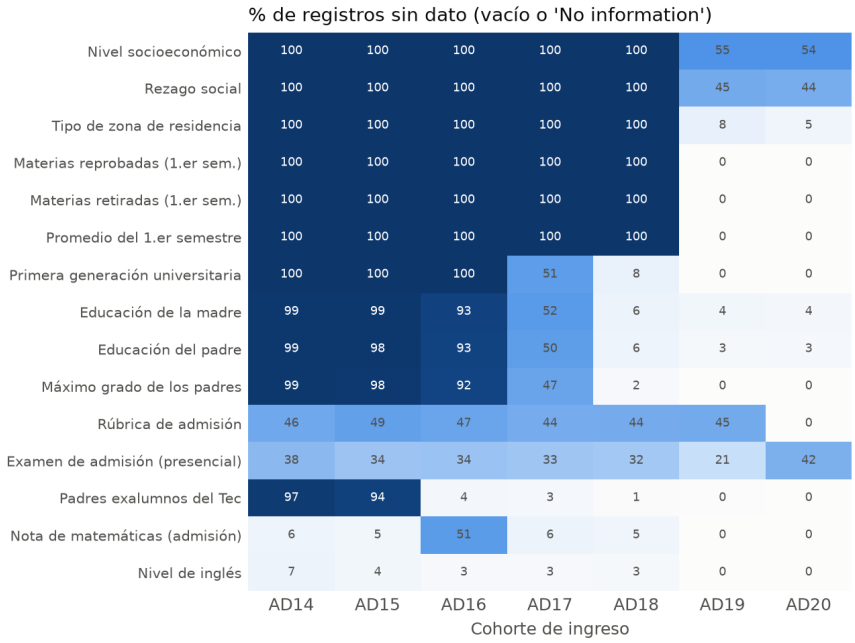



**Figura 1:** Porcentaje de registros sin dato por variable y cohorte (pregrado, conjunto de desarrollo). Los faltantes dependen de la cohorte: las notas del primer semestre solo existen en AD19 y AD20.

**Calidad de los datos.** El análisis exploratorio, disponible en el repositorio, encontró los problemas de la Tabla 1. El más relevante afecta a dos variables socioeconómicas, el nivel socioeconómico y el rezago social, que **no son utilizables**: falta el dato para cerca de la mitad de los estudiantes porque nunca se registró, la deserción es la misma con y sin dato, y cuando el dato

**Tabla 1:** Problemas de calidad encontrados en la población de estudio y decisión adoptada.

| Problema | Magnitud | Decisión |
|---|---|---|
| Nivel socioeconómico y rezago social sin dato; la deserción es igual con y sin dato | 55 % y 44 % | Se excluyen: no son utilizables. |
| Examen de admisión con dos escalas: presencial (400–1600) y en línea (0–100) | 570 estudiantes en línea | Se separa en dos variables antes de modelar. |
| Puntajes de admisión faltantes; quienes no los tienen desertan más | 22–32 % | Se rellenan con la mediana y se agrega una variable que indica que faltaban. |
| % total de apoyo económico, que es la suma de beca y crédito | $\rho = 0{,}96$ | Se excluye por repetir información. |
| Actividades extracurriculares medidas *durante* el año | 9 variables | Se excluyen: reflejan la permanencia en lugar de anticiparla. |
| Estudiantes que aparecen más de una vez | 6 casos | Cada estudiante queda en un solo lado de cada partición. |

existe casi no ayuda a distinguir a quienes desertan. Rellenar la mitad de una variable que no aporta solo agrega ruido, así que se excluyeron. La información familiar que sí está completa, la educación de los padres, se conservó.

**Variables construidas y selección.** Se creó una variable nueva, la *brecha de promedios*: promedio del primer semestre menos promedio de bachillerato. Indica si el estudiante bajó o subió su rendimiento al llegar a la universidad. Quienes bajan más de siete puntos desertan un 18 %; quienes mantienen su nivel, un 5 %. Esta variable mejoró a LightGBM en las cinco particiones de validación. Se probó otra (materias reprobadas más retiradas), pero no aportó y se descartó. Después se midió cuánto empeora el modelo si se desordenan al azar los valores de cada variable (importancia por permutación), siempre con los datos de entrenamiento. Cinco variables no aportaban nada (tipo de zona, examen en línea, procedencia, colegio del Tec y primera generación) y se retiraron; en particiones nuevas, el modelo obtuvo el mismo resultado sin ellas (PR-AUC de 0,243). El modelo final usa 23 variables: 13 numéricas (promedios de bachillerato y del semestre, brecha, materias reprobadas y retiradas, examen, rúbrica y nota de matemáticas de admisión, nivel de inglés, porcentajes de beca y de crédito, edad y carga académica) y 10 categóricas (educación del padre y de la madre, máximo grado de los padres, padres exalumnos, costo del colegio, tipo de beca, escuela, programa, región y género). La Tabla 2 resume el tamaño de los datos.

**Tabla 2:** Tamaño de los conjuntos de datos. El conjunto de prueba se reserva para la entrega final.

| Conjunto | Estudiantes | Desertores | Deserción | Variables |
|---|---|---|---|---|
| Desarrollo (80 %) | 19.408 | 1.505 | 7,8 % | 23 |
| Prueba sellada (20 %) | 4.843 | 367 | 7,6 % | 23 |

## 2. Flujo reproducible sin fuga de datos

Para que los resultados reflejen cómo funcionaría el modelo con estudiantes nuevos, se tomaron tres precauciones.

**1. Conjunto de prueba sellado.** Antes de cualquier análisis se apartó el 20 % de los estudiantes. Ese grupo no se ha mirado, ni siquiera en el análisis exploratorio, y solo se usará una vez, en la entrega final. Conserva la misma proporción de desertores y de cohortes, y cada estudiante queda en un solo lado.

**2. Preparación dentro de un `Pipeline`.** Todos los pasos van encadenados en este orden: (1)

rellenar los datos faltantes con la mediana y marcar, con una variable adicional, que faltaban; (2) estandarizar las variables numéricas; (3) convertir las categóricas en columnas de ceros y unos (*one-hot*), agrupando las categorías con menos de 30 estudiantes; (4) balancear las clases con SMOTE, si la validación lo elige, y (5) entrenar el modelo. La clave es que cada paso aprende solo de los datos de entrenamiento de cada partición (la mediana, la media, las categorías) y luego aplica lo aprendido, sin recalcular, a los datos de validación. Si la estandarización usara la media de todos los datos, el modelo ya tendría una pista de los estudiantes con que se evalúa. El orden también importa: se estandariza antes de SMOTE porque SMOTE crea estudiantes sintéticos a partir de los más parecidos, y para medir parecido las variables deben estar en la misma escala. Sin ese paso, el examen de admisión (hasta 1.600 puntos) pesaría mucho más que el porcentaje de beca (hasta 1). La regresión logística y la SVM también necesitan esa escala común; los árboles no, porque solo comparan valores dentro de una misma variable, y por eso CatBoost recibe los datos sin transformar. SMOTE actúa solo al entrenar, así que la validación conserva la proporción real de desertores. Pruebas automáticas del repositorio verifican que estos pasos aprenden únicamente de los datos de entrenamiento.

**3. Validación cruzada anidada.** Los datos de desarrollo se dividieron en cinco partes, sin repartir a un mismo estudiante entre dos partes y conservando la proporción de desertores. En cada una de cinco rondas, el modelo se entrena con cuatro partes y se evalúa con la quinta. Las decisiones (qué hiperparámetros usar y a partir de qué probabilidad emitir una alerta) se toman con nuevas divisiones internas de esas cuatro partes. Así, la parte con que se evalúa nunca influye en ninguna decisión. Los resultados se reportan como promedio y desviación estándar de las cinco rondas.

**Efecto de la fuga en el procedimiento del artículo.** Villar y Andrade aplican SMOTE a todos los datos antes de separar entrenamiento y validación. Para medir el efecto se entrenó el mismo modelo, LightGBM, de las dos maneras (Tabla 3). Con el procedimiento del artículo, el F1 de la clase deserción es de 0,95; con el nuestro, de 0,12. La razón es sencilla: SMOTE crea desertores sintéticos mezclando desertores reales. Si se crean antes de separar, los desertores de validación son mezclas de estudiantes que el modelo ya vio, y acertar es fácil. Esto sugiere que parte del alto desempeño del artículo (F1 de 0,83 a 0,88) viene del procedimiento y no del modelo.

**Tabla 3:** Mismo modelo evaluado con los dos procedimientos (cinco rondas). El PR-AUC se explica en la Sección 4.

| Protocolo | F1 deserción (umbral 0,5) | PR-AUC |
|---|---|---|
| SMOTE antes de separar (artículo) | 0,95 | 0,98 |
| SMOTE solo en entrenamiento (este trabajo) | 0,12 | 0,24 |

## 3. Reproducción y adaptación del método

El método del artículo compara árboles y ensambles clásicos con cuatro algoritmos de *boosting*, equilibra las clases con SMOTE (Chawla et al., 2002), ajusta los hiperparámetros e interpreta el mejor modelo con SHAP (Lundberg y Lee, 2017). La Tabla 4 resume qué se conservó, qué se cambió y por qué.

**Tabla 4:** Reproducción y adaptación del método de Villar y Andrade (2024).

| Elemento | Artículo | Este trabajo |
|---|---|---|
| Información | Ingreso + desempeño de los primeros semestres | Ingreso + desempeño del primer semestre (cohortes AD19–AD20) |
| Variable objetivo | Tres clases: graduado, rezagado y desertor | Dos clases: deserta o no en el primer año (el dataset no registra la graduación) |
| Modelos | Árbol, Random Forest, SVM, Gradient Boosting, XGBoost, LightGBM, CatBoost | Los mismos, más la regresión logística como referencia |
| Balanceo de clases | SMOTE sobre todos los datos | SMOTE solo sobre el entrenamiento, y como una opción más: la validación decide si se usa |
| Hiperparámetros | Optuna | `GridSearchCV`, herramienta del curso, dentro de la validación anidada |
| Métrica principal | F1 ponderado | PR-AUC, más *recall* y precisión de la clase deserción |
| Comparación | Prueba t pareada | Prueba t corregida de Nadeau y Bengio (2003) |
| Interpretación | SHAP | SHAP, repetido en cada ronda para comprobar su estabilidad |

**Modelos comparados.** Se compararon nueve modelos. Dos son líneas base: uno que asigna a todos la misma probabilidad de desertar y un árbol de decisión sin poda. La regresión logística sirve de referencia, porque es el modelo de Machine Learning I. Los otros seis son los del artículo: Random Forest, SVM con kernel RBF, Gradient Boosting, XGBoost, LightGBM y CatBoost. CatBoost trabaja directamente con las variables categóricas, sin convertirlas en ceros y unos; como SMOTE no funciona con ese tipo de variables, en su caso el desbalance se trata dando más peso a los desertores. Para que la SVM fuera viable con casi 20.000 estudiantes, se usó una aproximación de su kernel (Nystroem).

**Ajuste de hiperparámetros.** Cada modelo tiene una rejilla de hiperparámetros que controlan qué tan complejo puede ser (Tabla 5). En cada ronda, `GridSearchCV` probó todas las combinaciones, de 4 a 24 según el modelo, y eligió la de mayor PR-AUC. Usar o no SMOTE se trató como una opción más de la rejilla, y el resultado es claro: la validación **no eligió SMOTE en ninguna de las 30 decisiones** (seis modelos por cinco rondas), y CatBoost tampoco eligió dar más peso a los desertores. Crear desertores sintéticos no ayuda a ordenar a los estudiantes por riesgo; el desbalance se maneja mejor al fijar el umbral de alerta (Sección 4). Las configuraciones elegidas son simples: árboles poco profundos y regularización fuerte. La Figura 2 explica por qué. Cuando LightGBM puede usar árboles más grandes, su resultado en entrenamiento sube de 0,24 a 0,91, pero en validación solo mejora hasta las 8 hojas por árbol (0,250) y luego empeora: el modelo empieza a memorizar casos en lugar de aprender patrones. En la SVM, los valores elegidos quedaron en el borde de la rejilla, pero la curva muestra que ampliar $C$ a 1 ya no mejora (0,232 frente a 0,235). Por su alto costo (Sección 4), la rejilla de CatBoost se limitó a la profundidad y al peso de los desertores.

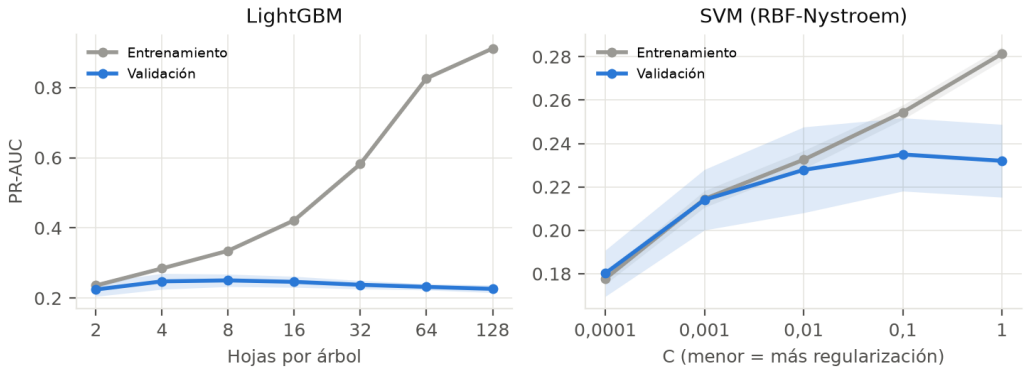



**Figura 2:** Curvas de validación alrededor de la configuración elegida (media ± desviación estándar de cinco rondas). Cuando la distancia entre entrenamiento y validación crece, el modelo está memorizando.

**Tabla 5:** Hiperparámetros explorados y combinación elegida con más frecuencia (entre paréntesis, en cuántas de las cinco rondas se eligió).

| Modelo | Valores explorados | Elegido (rondas) |
|---|---|---|
| Regresión logística | $C$: 0,0001, 0,0003, 0,001, 0,003, 0,01, 0,03, 0,1; balanceo: ninguno, SMOTE | 0,03, ninguno (2/5) |
| Random Forest | mín. registros por hoja: 5, 15, 40, 100; variables por división: $\sqrt{p}$, 0,4; balanceo: ninguno, SMOTE | 15, 0,4, ninguno (2/5) |
| SVM (kernel RBF) | $\gamma$: 0,001, 0,003, 0,01; $C$: 0,001, 0,01, 0,1; balanceo: ninguno, SMOTE | 0,01, 0,1, ninguno (5/5) |
| Gradient Boosting | tasa de aprendizaje: 0,02, 0,05; máx. hojas: 4, 8, 16; regularización L2: 0, 1; balanceo: ninguno, SMOTE | 0,05, 4, 1, ninguno (4/5) |
| XGBoost | profundidad máx.: 2, 3, 4; tasa de aprendizaje: 0,02, 0,05; peso mín. por hoja: 1, 10; balanceo: ninguno, SMOTE | 2, 0,05, 1, ninguno (4/5) |
| LightGBM | hojas: 4, 8, 16; tasa de aprendizaje: 0,02, 0,05; mín. registros por hoja: 20, 100; balanceo: ninguno, SMOTE | 4, 0,02, 100, ninguno (2/5) |
| CatBoost | profundidad: 4, 6; pesos de clase: ninguno, balanceados | 4, ninguno (4/5) |

**Otros elementos del artículo.** (i) *SMOTE frente a ADASYN.* Como en el artículo, con un árbol de decisión SMOTE fue algo mejor que ADASYN (F1 de la clase deserción de 0,154 frente a 0,148), pero el árbol sin balanceo superó a ambos (0,162). (ii) *Isolation Forest.* El artículo lo usa para ver si los desertores son estudiantes atípicos. Aquí tampoco lo son: el método casi no los distingue (ROC-AUC de 0,55, cuando 0,5 equivale al azar). (iii) *Regresión logística.* El artículo la descartó porque varias variables repetían información (multicolinealidad). Aquí ocurrió lo mismo, y la causa era concreta: el porcentaje total de apoyo es la suma exacta de beca y crédito. Al quitar ese total, el problema desaparece (el VIF de la beca baja de $10^7$ a 1,3) y el modelo se pudo conservar. La brecha de promedios también es la resta de dos variables del modelo, pero se dejó a propósito: a los árboles les sirve, porque analizan una variable a la vez y no pueden calcular esa resta por sí solos; a la regresión logística no le aporta, pero su regularización evita que la repetición la desestabilice.

**Por qué no se usaron modelos de densidad ni de clustering.** La Primera entrega los dejó para más adelante, según el calendario del curso, pero al final no se incluyeron, por tres razones. Primera, la guía del proyecto pide comparar modelos según el artículo, y el artículo solo usa un método de este tipo, Isolation Forest, que sí se replicó. Segunda, ese resultado muestra que los desertores no forman un grupo aparte ni atípico en los datos, que es justo lo que un enfoque de densidad necesitaría para funcionar. Tercera, el problema es supervisado: el clustering agrupa estudiantes sin mirar si desertaron, y nada garantiza que esos grupos separen a quienes se van. Además, modelos como las mezclas gaussianas suponen variables continuas con forma de campana, y aquí 10 de las 23 variables son categóricas.

## 4. Validación, métricas e interpretación preliminar

**Métricas.** Con solo 7,8 % de desertores, el F1 ponderado que usa el artículo engaña: un modelo que no alerta a nadie obtiene 0,89 sin encontrar a un solo desertor. Por eso los modelos se compararon con el PR-AUC, que mide qué tan bien ordenan a los estudiantes de mayor a menor riesgo (el azar obtiene 0,078 y un modelo perfecto, 1). También se reportan el *recall*, la parte de los desertores que se detecta, y la precisión, la parte de las alertas que acierta. La alerta se emite cuando la probabilidad estimada supera un umbral, que se eligió con los datos de entrenamiento para maximizar F2, una medida que da al *recall* el doble de peso que a la precisión. Así se traduce la prioridad fijada en la Primera entrega: dejar pasar a un desertor cuesta más que acompañar a un estudiante que no lo necesitaba.

**Resultados.** La Tabla 6 y la Figura 3 resumen el desempeño. Todos los modelos avanzados superan con claridad a las líneas base. El árbol sin poda queda casi al nivel del azar (0,091 frente a 0,078) porque memoriza los datos de entrenamiento. Las líneas base aparecen con *recall* de 1,00 porque dan a todos la misma probabilidad (o casi), así que el umbral termina alertando a todos; por eso se comparan por PR-AUC. Entre los modelos avanzados, los de *boosting* encabezan y están prácticamente empatados: CatBoost (0,248), XGBoost (0,248), LightGBM (0,247) y Gradient Boosting (0,245). Les siguen la SVM (0,235), Random Forest (0,230) y la regresión logística (0,224). Que todos ronden una ROC-AUC de 0,72 confirma lo que mostró el análisis exploratorio: los datos contienen una señal real, pero débil y repartida entre muchas variables.

**Tabla 6:** Resultados de la validación cruzada anidada: PR-AUC y ROC-AUC (promedio ± desviación estándar de las cinco rondas); *recall*, precisión y proporción de estudiantes alertados con el umbral elegido. En negrita, el mejor PR-AUC.

| Modelo | PR-AUC | ROC-AUC | Recall | Precisión | Alertas |
|---|---|---|---|---|---|
| Mayoría | 0,078 ±0,000 | 0,500 ±0,000 | 1,00 | 0,08 | 1,00 |
| Árbol sin poda | 0,091 ±0,005 | 0,546 ±0,011 | 1,00 | 0,08 | 1,00 |
| Regresión logística | 0,224 ±0,019 | 0,715 ±0,010 | 0,64 | 0,14 | 0,36 |
| Random Forest | 0,230 ±0,018 | 0,697 ±0,012 | 0,58 | 0,14 | 0,33 |
| SVM (kernel RBF) | 0,235 ±0,017 | 0,716 ±0,007 | 0,63 | 0,14 | 0,36 |
| Gradient Boosting | 0,245 ±0,022 | 0,716 ±0,007 | 0,68 | 0,13 | 0,40 |
| XGBoost | 0,248 ±0,021 | 0,717 ±0,009 | 0,60 | 0,14 | 0,33 |
| LightGBM | 0,247 ±0,023 | 0,716 ±0,005 | 0,67 | 0,13 | 0,40 |
| CatBoost | **0,248 ±0,023** | 0,711 ±0,009 | 0,66 | 0,13 | 0,39 |

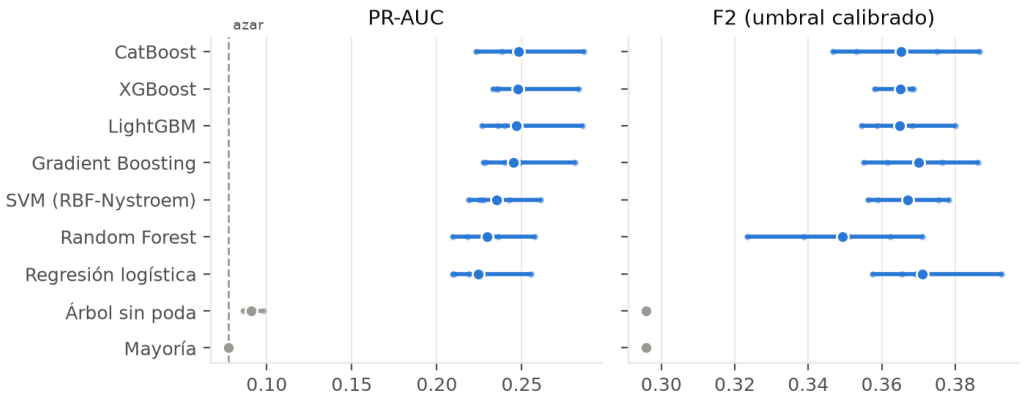



**Figura 3:** PR-AUC y F2 de cada modelo en las cinco rondas (puntos pequeños) y su promedio (punto grande). La línea punteada es el PR-AUC de un modelo al azar.

**¿Son reales las diferencias?** Las cinco rondas comparten buena parte de sus datos de en-

trenamiento, así que sus resultados no son independientes. La prueba t corregida de Nadeau y Bengio (2003) tiene en cuenta ese solapamiento; la prueba t pareada del artículo, no. Con la prueba corregida, CatBoost no se distingue de XGBoost, LightGBM ni Gradient Boosting ($p \geq 0{,}44$), ni con claridad de la SVM ($p = 0{,}09$) o de Random Forest ($p = 0{,}13$); sí supera a la regresión logística ($p = 0{,}026$) y a las líneas base ($p < 0{,}001$). Con la prueba sin corregir, las diferencias con la SVM y Random Forest habrían parecido reales ($p = 0{,}030$ y $0{,}047$). En resumen, el *boosting* mejora a un buen modelo lineal, pero poco.

**Costo computacional.** Si varios modelos empatan, conviene preferir el más barato (Tabla 7). CatBoost, primero por un margen mínimo, es el más lento: tarda 16 segundos en entrenarse y su búsqueda de hiperparámetros tomó 30 minutos con solo cuatro combinaciones. LightGBM logra el mismo resultado entrenándose en medio segundo y probando 24 combinaciones en 5,5 minutos. Random Forest ocupa 27 MB, frente a menos de 0,4 MB de los modelos de *boosting*. Predecir es rápido con todos: menos de 16 milisegundos por cada mil estudiantes.

**Tabla 7:** Costo computacional (procesador de 12 hilos). Búsqueda: validación cruzada anidada completa. Ajuste y predicción: modelo final con los hiperparámetros elegidos, sobre los 19.408 estudiantes de desarrollo (mediana de tres repeticiones).

| Modelo | PR-AUC | Combinaciones | Búsqueda (min) | Ajuste (s) | Predicción (ms/1.000) | Tamaño (MB) |
|---|---|---|---|---|---|---|
| Mayoría | 0,078 | 1 | 0,4 | 0,20 | 2,9 | 0,01 |
| Árbol sin poda | 0,091 | 1 | 0,3 | 0,44 | 2,9 | 0,32 |
| Regresión logística | 0,224 | 14 | 1,5 | 0,30 | 3,4 | 0,01 |
| Random Forest | 0,230 | 16 | 24,9 | 4,57 | 14,9 | 26,90 |
| SVM (kernel RBF) | 0,235 | 18 | 3,9 | 2,35 | 15,7 | 2,31 |
| Gradient Boosting | 0,245 | 24 | 6,0 | 1,23 | 9,2 | 0,23 |
| XGBoost | 0,248 | 24 | 11,8 | 1,02 | 5,6 | 0,37 |
| LightGBM | 0,247 | 24 | 5,5 | 0,55 | 7,1 | 0,25 |
| CatBoost | 0,248 | 4 | 29,7 | 15,96 | 5,4 | 0,21 |

**Qué significa en la práctica.** Con el umbral elegido, CatBoost detecta al 66 % de los desertores, pero alerta al 39 % de los estudiantes, y solo 13 de cada 100 alertas son desertores reales. Es más útil pensarlo así: si la universidad solo puede acompañar al 20 % de estudiantes con mayor riesgo, el modelo le permite llegar al 46 % de quienes van a desertar, 2,3 veces más que eligiendo al azar. Si acompaña al 10 %, llega al 32 %, y una de cada cuatro alertas es correcta.

**Variables más influyentes.** Para saber en qué se basa el modelo se usó SHAP, que reparte cada predicción entre las variables. Se calculó en las cinco rondas para comprobar que las conclusiones no cambian con los datos. El promedio del primer semestre fue la variable más influyente en cuatro rondas, y el porcentaje de beca en la restante; después vienen el promedio de bachillerato, la rúbrica de admisión, el programa y la nota de matemáticas. La Figura 4 muestra en qué dirección actúa cada una: un promedio del semestre bajo sube mucho el riesgo, mientras que uno alto lo baja poco; las becas altas y un buen desempeño previo lo reducen; tener más edad y bajar el rendimiento respecto al bachillerato lo aumentan. Es el mismo hallazgo central del artículo: el desempeño académico temprano es el mejor predictor de la deserción. Son asociaciones útiles para predecir, no causas.

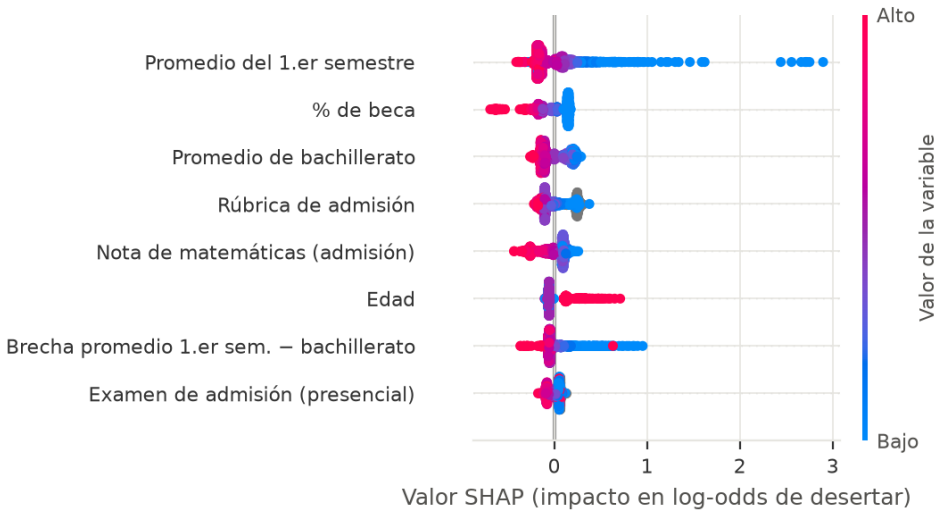



**Figura 4:** Valores SHAP de CatBoost para las variables numéricas más influyentes (2.000 estudiantes de validación de la última ronda). Cada punto es un estudiante; los valores positivos aumentan el riesgo estimado de desertar.

**Errores e implicaciones.** El modelo casi no detecta a los desertores con beca por talento académico (6 de cada 100): como grupo tienen bajo riesgo, y lo que los lleva a irse no está en los datos. En cambio, alerta al 59 % de los estudiantes sin beca que no desertan. Por eso el modelo sirve para decidir a quién acompañar primero, no para etiquetar a los estudiantes, ya que varias de sus señales reflejan la situación económica.

**Pasos para la entrega final.** Evaluar una sola vez el modelo elegido, con su umbral ya fijado, en el conjunto de prueba; profundizar la interpretación, y desplegarlo en una aplicación de Streamlit que explique cada predicción.

**Código y uso de IA.** El análisis se hizo en Python con scikit-learn (Pedregosa et al., 2011), imbalanced-learn (Lemaître et al., 2017), XGBoost (Chen y Guestrin, 2016), LightGBM (Ke et al., 2017), CatBoost (Prokhorenkova et al., 2018) y SHAP. Se utilizó un asistente de inteligencia artificial (Claude, de Anthropic) como apoyo en la escritura del código y la revisión del texto; los autores verificaron los resultados y pueden sustentar cada decisión técnica.

## Referencias

[1] Alvarado-Uribe, J., Mejía-Almada, P., Masetto Herrera, A. L., Molontay, R., Hilliger, I., Hegde, V., Montemayor Gallegos, J. E., Ramírez Díaz, R. A., & Ceballos, H. G. (2022). Student Dataset from Tecnologico de Monterrey in Mexico to Predict Dropout in Higher Education. *Data*, 7(9), 119. https://doi.org/10.3390/data7090119

[2] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.

[3] Chen, T. & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. En *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794).

[4] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30.

[5] Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, 18(17), 1–5.

[6] Lundberg, S. M. & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30.

[7] Nadeau, C. & Bengio, Y. (2003). Inference for the generalization error. *Machine Learning*, 52(3), 239–281.

[8] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[9] Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: unbiased boosting with categorical features. *Advances in Neural Information Processing Systems*, 31.

[10] Villar, A. & Robledo Velini de Andrade, C. (2024). Supervised machine learning algorithms for predicting student dropout and academic success: a comparative study. *Discover Artificial Intelligence*, 4:2. https://doi.org/10.1007/s44163-023-00079-z

## Anexo: código y reproducibilidad

El código está en https://github.com/miguel-camargo611/ML2-desercion-estudiantil. Para repetir todos los resultados de este informe:

1. Instalar el entorno con `pip install -e .` (las versiones de las librerías están fijadas).
2. Solicitar el dataset al TEC DataHub y copiar el archivo de Excel recibido en la carpeta `data/raw/`.
3. Ejecutar `desercion all`.

Todo el flujo usa la semilla 42, de modo que se obtienen las mismas cifras. El análisis exploratorio completo está en `notebooks/01_exploracion_datos.ipynb`, y las pruebas automáticas (`pytest`) verifican, con datos sintéticos, que la preparación de los datos no filtra información de validación.